

AI Resume Analyzer

A Beginner's Complete Guide to the Project's Code, Design, and Purpose

This document walks through every file in the AI Resume Analyzer project, explains exactly what each piece of code does and why it was written that way, identifies the specific programming and data-science skills demonstrated in each file, and closes with a full explanation of the project's purpose and its value as a learning project for someone new to Natural Language Processing, Streamlit, and applied machine learning.

Who this guide is for: complete beginners to NLP, Python web apps, and data science projects. No prior experience assumed.

Table of Contents

1. Project Overview
2. Project Folder Structure
3. File: data/skills_list.py — The Skills Reference List
4. File: utils/extract_text.py — PDF Text Extraction
5. File: utils/analyzer.py — The Core Analysis Engine
6. File: app.py — The Streamlit User Interface
7. Skills & Concepts Demonstrated (Summary)
8. The Real Motive Behind This Project
9. A Beginner's Roadmap: How to Read This Project

1. Project Overview

The AI Resume Analyzer is a web application, built entirely in Python using the Streamlit framework, that allows a user to upload a resume in PDF format and paste in a job description. The application then reads the resume, compares its content against the job description using two different Natural Language Processing techniques, checks the resume's structure and writing quality the way a professional Applicant Tracking System (ATS) would, and produces a full visual report: a match score, a breakdown of matched and missing skills, resume health metrics, and specific, actionable suggestions for improvement.

The project is intentionally built without relying on any external paid AI service such as OpenAI or Gemini. Every piece of intelligence in the app — from reading the skills in a resume to scoring how well it matches a job — is generated using open-source, locally-run Python libraries. This was a deliberate design decision (explained further in Section 8) that makes the project reliable, free to run, and fully self-contained.

The project is organized using good software engineering practice: the code is split across multiple files, each with a single, clear responsibility, rather than being written as one long script. This is called **separation of concerns**, and it is one of the most important habits a beginner programmer can learn early on.

2. Project Folder Structure

Before looking at individual files, it helps to see how they all fit together. Below is the structure of the whole project:

```
resume-analyzer/  
■  
■■■ app.py                <- Main Streamlit application (the UI)  
■■■ requirements.txt      <- List of installed Python packages  
■  
■■■ data/  
■   ■■■ __init__.py       <- Makes 'data' an importable package  
■   ■■■ skills_list.py    <- Master list of known skills  
■  
■■■ utils/  
    ■■■ __init__.py       <- Makes 'utils' an importable package  
    ■■■ extract_text.py   <- Reads text out of an uploaded PDF  
    ■■■ analyzer.py       <- All scoring, NLP and analysis logic
```

Notice the flow of information through this structure: **app.py** is the only file the user ever sees running — it is the 'front door' of the application. It never does any heavy analysis itself. Instead, it calls functions from **extract_text.py** and **analyzer.py**, which do the real work, and **analyzer.py** in turn pulls its reference data from **skills_list.py**. This layered design means each file can be tested, understood, and modified on its own without needing to touch the others.

3. File: data/skills_list.py

The Skills Reference List

```
SKILLS_LIST = [  
    # Programming languages  
    "python", "java", "javascript", "typescript", "c++", "c#", "c",  
    "go", "golang", "rust", "ruby", "php", "swift", "kotlin", "scala",  
    "r", "matlab", "perl", "sql", "bash", "shell scripting",  
  
    # Web development  
    "html", "css", "sass", "react", "angular", "vue", "next.js",  
    "node.js", "express", "django", "flask", "fastapi", "spring boot",  
    "asp.net", "rest api", "graphql", "api development", "webpack",  
    "streamlit", "bootstrap", "tailwind css",  
  
    # Data science / ML / AI  
    "machine learning", "deep learning", "artificial intelligence",  
    "nlp", "natural language processing", "computer vision",  
    "data analysis", "data science", "data visualization",  
    "data engineering", "data mining", "statistics", "predictive modeling",  
    "pandas", "numpy", "scikit-learn", "tensorflow", "pytorch", "keras",  
    "opencv", "matplotlib", "seaborn", "plotly", "spacy", "nltk",  
    "hugging face", "transformers", "llm", "large language models",  
    "feature engineering", "model deployment", "mlops", "a/b testing",  
  
    # Databases, Cloud & DevOps, Tools, Project Management,  
    # Soft Skills, Marketing/Design, and General Office skills  
    # (full list continues in the same pattern...)  
]
```

What this code does:

This file defines a single Python variable, **SKILLS_LIST**, which is simply a list of strings. Each string is the name of a skill — a programming language, a framework, a soft skill, or a tool — written entirely in lowercase. This list acts as the project's 'vocabulary': it is the master reference the rest of the application checks resumes and job descriptions against. The list is organized into labelled groups (programming languages, web development, data science, databases, cloud/DevOps, tools, project management, soft skills, marketing/design, and general office skills) purely for human readability — Python treats it as one single, flat list regardless of the comments.

Skills and qualities demonstrated:

This file might look simple, but it demonstrates an important data science principle called **building a domain-specific taxonomy** — creating a structured, curated vocabulary for a specific problem area (in this case, employment skills) rather than trying to detect skills 'intelligently' with no reference point. This is a completely standard, real-world technique: many production keyword-matching and tagging systems (including some resume screening tools used by real companies) are built on carefully maintained keyword lists exactly like this one. It also shows an understanding of separating **data from logic** — the skills themselves live in their own file, separate from the code that uses them, so the list can be edited or expanded at any time without needing to touch or re-test the analysis code at all.

4. File: utils/extract_text.py

PDF Text Extraction

```
import pdfplumber

def extract_text_from_pdf(uploaded_file):
    text = ""
    with pdfplumber.open(uploaded_file) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
    return text
```

What this code does:

This file has exactly one job, and it does it well: turning an uploaded PDF file into plain, readable text that the rest of the program can work with. It uses the third-party library **pdfplumber**, which understands the internal structure of PDF files, to open the uploaded resume directly from memory (it does not need to be saved to disk first). The function then loops through every page in the document with **pdf.pages**, and for each page calls **.extract_text()** to pull out whatever readable text exists on that page. Every page's text is appended onto a single growing string called **text**, with a newline character added between pages to keep the content readable, and the completed string is returned once every page has been processed.

The line **if page_text:** is a small but important defensive check. Some PDF pages contain no extractable text at all — for example, a page that is really just a scanned image rather than real text — and in that case **.extract_text()** returns **None** instead of a string. Without this check, trying to add **None** to the existing text string would immediately crash the program with a **TypeError**. This one line is what is generally called **defensive programming**: anticipating a case that could go wrong and handling it gracefully before it ever causes a crash.

Skills and qualities demonstrated:

This file demonstrates **document parsing** (the general skill of extracting structured or unstructured information out of a file format not originally designed to be read by code), safe handling of external, untrusted input (an uploaded file could be anything — malformed, scanned, encrypted, or empty — and the function is written to not break on any of those cases), and the use of a well-established third-party library rather than attempting to parse the PDF binary format manually. Knowing when to use an existing, trusted library instead of reinventing a solution is itself a mark of good engineering judgement.

5. File: utils/analyzer.py

The Core Analysis Engine

This is the largest and most important file in the project — it contains every piece of intelligence the app has. To make it manageable, it is broken into small functions, each responsible for exactly one calculation or check. Below, each function is shown and explained individually, in the order they appear in the file.

Imports and setup

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer, util
from data.skills_list import SKILLS_LIST

semantic_model = SentenceTransformer('all-MiniLM-L6-v2')

REQUIRED_SECTIONS = ["experience", "education", "skills"]
RECOMMENDED_SECTIONS = ["summary", "projects", "certifications"]

STRONG_VERBS = [
    "led", "managed", "developed", "designed", "implemented", "built",
    "created", "launched", "optimized", "improved", "increased", "reduced",
    "achieved", "delivered", "architected", "spearheaded", "drove",
    "streamlined", "automated", "negotiated", "mentored"
]

WEAK_PHRASES = [
    "responsible for", "worked on", "helped with", "was involved in",
    "duties included", "tasked with"
]
```

This top section imports every tool the file needs: **re** is Python's built-in library for regular expressions (pattern matching in text); **TfidfVectorizer** and **cosine_similarity** come from scikit-learn and power the keyword-based matching score; **SentenceTransformer** and **util** come from the sentence-transformers library and power the meaning-based matching score. The line **semantic_model = SentenceTransformer(...)** loads a pre-trained transformer model **once**, when the file is first imported, rather than reloading it every time a resume is analyzed — this is a deliberate performance decision, since loading the model takes noticeable time and only needs to happen a single time per app session. The remaining lists (**REQUIRED_SECTIONS**, **STRONG_VERBS**, **WEAK_PHRASES**) are reference data used later by the resume-quality checking functions.

Function: clean_text()

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-z0-9\s+', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

This is a standard **text preprocessing** function, a fundamental step in almost every NLP pipeline. It lowercases all text (so 'Python' and 'python' are treated identically), then uses a regular expression to strip out anything that is not a letter, digit, or whitespace (removing punctuation, symbols, and special characters that would otherwise confuse a word-matching algorithm), and finally collapses any run of multiple spaces down to a single space and trims leading/trailing whitespace. This is exactly the kind of cleaning real-world NLP systems perform before doing any comparison or scoring.

Function: extract_skills()

```
def extract_skills(text):
    text = text.lower()
    found = [skill for skill in SKILLS_LIST if skill in text]
    return found
```

This function checks a given piece of text (either a resume or a job description) against every entry in **SKILLS_LIST** and returns only the skills that actually appear in it. The single line **[skill for skill in SKILLS_LIST if skill in text]** is a Python **list comprehension** — a compact way of writing a loop that builds a new list, here reading as 'give me every skill in the master list, but only if that skill's text is actually found inside the given text.' This is the engine behind both the 'Matched Skills' and 'Missing Skills' sections of the final report.

Function: `get_match_score()` — TF-IDF Similarity

```
def get_match_score(resume_text, jd_text):
    resume_clean = clean_text(resume_text)
    jd_clean = clean_text(jd_text)
    documents = [resume_clean, jd_clean]
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(documents)
    score = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
    return round(float(score[0][0]) * 100, 2)
```

This function calculates how similar the resume and job description are, purely based on shared, distinctive vocabulary — this is the first of the project's two NLP scoring techniques. Both texts are cleaned, then passed to a **TfidfVectorizer**, which converts each piece of text into a numerical vector. TF-IDF stands for **Term Frequency – Inverse Document Frequency**: it measures how important a word is to a specific document by weighing how often it appears in that document against how common it is across all documents, automatically giving less weight to generic filler words. Setting **stop_words='english'** also removes common words like 'the', 'and', and 'is' before this calculation even happens. Once both texts are represented as numeric vectors, **cosine_similarity** measures the angle between those two vectors — a smaller angle means more similar content — and the result, a number between 0 and 1, is converted into a more readable 0-to-100 percentage score.

Function: `get_semantic_score()` — Sentence Embedding Similarity

```
def get_semantic_score(resume_text, jd_text):
    embeddings = semantic_model.encode([resume_text, jd_text])
    similarity = util.cos_sim(embeddings[0], embeddings[1])
    return round(float(similarity[0][0]) * 100, 2)
```

This is the project's second, more advanced scoring technique, and it is what separates this project from a typical beginner keyword-matcher. Instead of comparing exact word overlap like TF-IDF does, this function uses a pre-trained **transformer neural network** (the same family of deep learning architecture behind modern large language models) to convert each piece of text into an **embedding** — a list of numbers that represents the actual meaning of the text, learned from being trained on huge amounts of real language. Two pieces of text that mean similar things, even if they use completely different words (for example, 'led a team of five engineers' and 'managed a small engineering staff'), will produce embeddings that are close together in this numerical space. **util.cos_sim** then measures that closeness, again using cosine similarity, and the result is scaled into a percentage. This function represents genuine applied deep learning, not just statistical word counting.

Function: `detect_sections()`

```
def detect_sections(resume_text):
    text_lower = resume_text.lower()
    found_required = [s for s in REQUIRED_SECTIONS if s in text_lower]
    found_recommended = [s for s in RECOMMENDED_SECTIONS if s in text_lower]
```



```

missing_required = [s for s in REQUIRED_SECTIONS if s not in found_required]
return {
    "found_required": found_required,
    "missing_required": missing_required,
    "found_recommended": found_recommended
}

```

Real Applicant Tracking Systems check whether a resume has proper, clearly labelled sections before they even look at the content, because a resume without standard headings can fail to parse correctly at all. This function checks the resume text for the presence of required section names ('experience', 'education', 'skills') and bonus, recommended sections ('summary', 'projects', 'certifications'), and returns a dictionary summarizing what was found and what is missing. This mirrors real-world ATS structural validation on a simplified scale.

Function: check_quantifiable_achievements()

```

def check_quantifiable_achievements(resume_text):
    number_pattern = r'\b\d+[\d,]*\.\d*\s*(%|percent|\$|million|thousand|k\b)?'
    matches = re.findall(number_pattern, resume_text.lower())
    quantified_count = len([m for m in matches if m])
    lines = resume_text.split('\n')
    bullet_lines = [l for l in lines if len(l.strip()) > 20]
    total_bullets = max(len(bullet_lines), 1)
    quantified_ratio = round((quantified_count / total_bullets) * 100, 1)
    return {
        "quantified_count": quantified_count,
        "quantified_ratio": min(quantified_ratio, 100)
    }

```

One of the most consistent pieces of advice from real recruiters and career coaches is that a resume should quantify achievements with real numbers — 'increased sales by 20%' is far stronger than 'responsible for increasing sales.' This function uses a regular expression to search for numeric patterns (numbers, percentages, dollar amounts, or terms like 'million' or 'thousand') throughout the resume, counts how many were found, and compares that count against the number of substantial lines in the resume (used as a rough proxy for the number of bullet points) to produce a 'quantified achievement ratio.' This function demonstrates practical use of regular expressions to extract structured signals (numbers) from unstructured text.

Function: check_action_verbs()

```

def check_action_verbs(resume_text):
    text_lower = resume_text.lower()
    strong_count = sum(1 for verb in STRONG_VERBS if verb in text_lower)
    weak_matches = [phrase for phrase in WEAK_PHRASES if phrase in text_lower]
    return {
        "strong_verb_count": strong_count,
        "weak_phrases_found": weak_matches
    }

```

This function checks the resume's writing style at the language level, counting how many confident, results-oriented action verbs (like 'led', 'built', 'optimized') are present, and separately flagging weak, passive phrasing (like 'responsible for' or 'helped with') that career advisors typically recommend avoiding. This is a simple but genuinely useful form of automated writing-quality analysis, built entirely from a curated reference list rather than a complex model.

Function: check_contact_info()

```

def check_contact_info(resume_text):
    issues = []
    has_email = bool(re.search(r'[\w.-]+@[ \w.-]+\.\w+', resume_text))
    has_phone = bool(re.search(
        r'(\+?\d{1,3}[-.\s]?)?(? \d{3})?[-.\s]\d{3}[-.\s]? \d{4}', resume_text
    ))
    has_linkedin = "linkedin" in resume_text.lower()

```

```

if not has_email:
    issues.append("No email address found.")
if not has_phone:
    issues.append("No phone number found.")
if not has_linkedin:
    issues.append("No LinkedIn profile mentioned.")
return {
    "has_email": has_email, "has_phone": has_phone,
    "has_linkedin": has_linkedin, "issues": issues
}

```

This function uses regular expressions to detect whether an email address and a phone number pattern exist anywhere in the resume, and does a simple text search for the word 'linkedin'. Missing contact information is one of the most common — and most costly — resume mistakes, since an otherwise excellent resume is useless to a recruiter who cannot find a way to make contact. This is a small function, but it checks something genuinely high-impact in practice.

Function: `student_specific_tips()`

```

def student_specific_tips(resume_text):
    tips = []
    text_lower = resume_text.lower()
    if "project" not in text_lower:
        tips.append("Consider adding a 'Projects' section.")
    if "certification" not in text_lower and "certificate" not in text_lower:
        tips.append("Consider adding relevant certifications.")
    if "gpa" not in text_lower:
        tips.append("If your GPA is strong (3.5+), consider including it.")
    return tips

```

This function tailors advice specifically for the project's intended primary audience — students and early-career job seekers — who often lack extensive work history but may have strong academic projects, certifications, or grades that are worth highlighting instead. This shows an understanding of designing for a specific user persona rather than building a generic, one-size-fits-all tool.

Function: `generate_suggestions()`

```

def generate_suggestions(missing_skills, score):
    suggestions = []
    if missing_skills:
        skills_str = ", ".join(missing_skills)
        suggestions.append(f"Consider highlighting: {skills_str}")
    if score < 40:
        suggestions.append("Low overlap. Use more of the JD's own keywords and phrasing.")
    elif score < 70:
        suggestions.append("Decent overlap, but room to better align language with the JD.")
    else:
        suggestions.append("Strong match! Your resume aligns well with this job description.")
    if len(missing_skills) > 5:
        suggestions.append("You're missing several key skills the JD asks for.")
    return suggestions

```

This function is simple, rule-based decision logic: it looks at the missing skills list and the overall match score, and returns a small set of tailored, human-readable suggestions using plain if/elif/else branching. Unlike the TF-IDF and semantic scoring functions, this is not machine learning — it is deterministic, hand-written logic — and that is a completely valid, common, and reliable design choice for producing feedback that is predictable and always makes sense, rather than relying on a language model that could occasionally produce an odd or unhelpful response.

Function: `analyze_resume()` — Tying Everything Together

```

def analyze_resume(resume_text, jd_text):
    try:
        resume_skills = extract_skills(resume_text)

```

```

jd_skills = extract_skills(jd_text)
matched = list(set(resume_skills) & set(jd_skills))
missing = list(set(jd_skills) - set(resume_skills))

tfidf_score = get_match_score(resume_text, jd_text)
semantic_score = get_semantic_score(resume_text, jd_text)
final_score = round((tfidf_score * 0.3) + (semantic_score * 0.7), 2)

sections = detect_sections(resume_text)
achievements = check_quantifiable_achievements(resume_text)
verbs = check_action_verbs(resume_text)
contact = check_contact_info(resume_text)

content_score = min(100, (achievements['quantified_ratio'] * 0.6)
                    + (verbs['strong_verb_count'] * 4))
format_score = 100 - (len(sections['missing_required']) * 25) \
                    - (len(contact['issues']) * 10)
format_score = max(format_score, 0)
keyword_score = final_score

overall_professional_score = round(
    (content_score * 0.35) + (format_score * 0.25) + (keyword_score * 0.4), 1
)

suggestions = generate_suggestions(missing, final_score)
# ...additional suggestions appended from each check...
suggestions.extend(student_specific_tips(resume_text))

return {
    "matched_skills": matched, "missing_skills": missing,
    "tfidf_score": tfidf_score, "semantic_score": semantic_score,
    "match_score": final_score,
    "overall_professional_score": overall_professional_score,
    "content_score": round(content_score, 1),
    "format_score": round(format_score, 1),
    "keyword_score": round(keyword_score, 1),
    "sections": sections, "achievements": achievements,
    "verbs": verbs, "contact": contact, "suggestions": suggestions
}
except Exception as e:
    return {"error": f"Something went wrong: {str(e)}"}

```

This is the orchestration function — the one all the others exist to support — and it is called directly by **app.py**. It calls every function described above in sequence, collects all their results, and combines them into the project's most distinctive design decision: a **three-pillar weighted scoring system** modeled on how real commercial ATS platforms actually score resumes. The **content_score** reflects writing quality (strong verbs and quantified achievements), the **format_score** reflects structural completeness (required sections and contact information present), and the **keyword_score** reflects relevance to the specific job (the blended TF-IDF and semantic similarity score). These three are combined with different weights (35%, 25%, and 40% respectively) into one overall professional score. The entire function is wrapped in a **try/except** block, so that if anything unexpected happens for any reason — an unusual resume format, an empty input, or any bug — the app returns a clean, readable error message instead of crashing entirely. This is a hallmark of production-quality, user-facing code.

6. File: app.py

The Streamlit User Interface

This file is the only one that actually runs when the app is launched. Its job is entirely presentational: it collects input from the user, calls the analysis functions from **analyzer.py** and **extract_text.py**, and displays the results attractively. It deliberately contains no analysis logic of its own — this separation is what makes the project maintainable.

Setup and page configuration

```
import streamlit as st
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from wordcloud import WordCloud
```

```
from utils.extract_text import extract_text_from_pdf
from utils.analyzer import analyze_resume
```

```
st.set_page_config(page_title="AI Resume Analyzer", page_icon="■", layout="wide")
st.title("■ AI Resume Analyzer")
st.write("Upload your resume and a job description to see how well they match.")
```

The imports bring in Streamlit itself, plus three visualization libraries: **matplotlib** for the skills bar chart, **plotly** for the interactive gauge chart, and **wordcloud** for the resume keyword cloud. It then imports the two custom functions this file relies on. **st.set_page_config(...)** sets the browser tab's title and icon and switches the layout to 'wide' mode so the multi-column results have room to breathe.

Visualization functions

```
def plot_gauge(score):
    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=score,
        title={'text': "Overall Match Score"},
        gauge={
            'axis': {'range': [0, 100]},
            'bar': {'color': "#1f77b4"},
            'steps': [
                {'range': [0, 40], 'color': "#f8d7da"},
                {'range': [40, 70], 'color': "#fff3cd"},
                {'range': [70, 100], 'color': "#d4edda"}
            ]
        }
    ))
    st.plotly_chart(fig, use_container_width=True)
```

This function builds an interactive speedometer-style gauge using Plotly, with color-coded zones (red for a weak score, yellow for a moderate score, green for a strong score) so the user can instantly gauge (quite literally) their resume's match quality without needing to interpret a raw number. Two similar helper functions, **plot_skills_chart()** and **plot_wordcloud()**, follow the same pattern using matplotlib and the wordcloud library respectively — each builds a chart object and hands it to Streamlit to render inline in the browser.

The main application flow

```
resume_file = st.file_uploader("Upload your resume (PDF only)", type=["pdf"])
jd_text = st.text_area("Paste the Job Description here", height=200)
analyze_clicked = st.button("Analyze Resume")
```

```
if analyze_clicked:
    if not resume_file:
        st.error("Please upload a resume PDF first.")
```

```

elif not jd_text.strip():
    st.error("Please paste a job description.")
else:
    with st.spinner("Reading your resume..."):
        resume_text = extract_text_from_pdf(resume_file)

    if not resume_text.strip():
        st.error("Couldn't extract text from this PDF. Try a different file.")
    else:
        with st.spinner("Analyzing..."):
            result = analyze_resume(resume_text, jd_text)

            if "error" in result:
                st.error(result["error"])
            else:
                # Display gauge, health breakdown, skills, charts, suggestions...
                plot_gauge(result['match_score'])
                # ...(full display code continues)

```

This is the heart of the user interface. **st.file_uploader** and **st.text_area** create the input widgets and store whatever the user provides in **resume_file** and **jd_text**. Nothing happens until **analyze_clicked** becomes true, which only occurs on the frame after the user presses the 'Analyze Resume' button — this reflects an important quirk of Streamlit's execution model: the entire script re-runs from top to bottom every time the user interacts with any widget on the page, and code is written to respond conditionally to that.

Notice the layered validation: first it checks whether a file was uploaded at all, then whether the job description box is empty, then — after extracting the PDF's text — whether that extraction actually produced any usable text (protecting against PDFs that are just scanned images with no selectable text). Only once every one of these checks passes does the code proceed to call **analyze_resume(...)**. This layered 'guard clause' pattern is a widely used, professional way of handling user input safely, giving a specific, helpful error message at exactly the point where something is missing, rather than letting the program crash with a cryptic technical error.

7. Skills & Concepts Demonstrated (Summary)

The table below summarizes every distinct technical skill and concept demonstrated across the whole project, for quick reference — useful for a resume, portfolio description, or interview talking points.

Category	Specific Skill / Concept Demonstrated
File & PDF Handling	Binary file parsing with pdfplumber; defensive None-checking
Text Preprocessing	Lowercasing, regex-based cleaning, whitespace normalization
Classical NLP	TF-IDF vectorization; cosine similarity
Deep Learning / NLP	Sentence-transformer embeddings for semantic similarity
Regular Expressions	Pattern matching for numbers, emails, phone numbers
Rule-Based Systems	Weighted multi-factor scoring; conditional suggestion logic
Software Architecture	Separation of concerns across modules; reusable functions
Error Handling	try/except wrapping to prevent crashes on unexpected input
Data Visualization	Plotly gauges, Matplotlib bar charts, Word Clouds
Web App Development	Streamlit widgets, state/session flow, conditional rendering
UX / Design Thinking	Layered input validation; persona-specific (student) tips

8. The Real Motive Behind This Project

It is worth stepping back from the code and asking directly: why does this project exist, and what problem is it actually solving?

Job searching is a numbers game that disproportionately punishes candidates — especially students and early-career job seekers — who don't yet know the unwritten rules of resume writing. A brilliant candidate can be filtered out before a human ever reads their resume, simply because their formatting confuses an automated screening system, their writing uses passive language instead of confident action verbs, or their resume happens to use different words than the job posting even though they have the exact skills being asked for. Large companies and recruiters have relied on Applicant Tracking Systems for years to manage the overwhelming volume of applications they receive, but individual job seekers — particularly students without access to expensive career-coaching services — have historically had no way to see their resume the way these automated systems see it.

This project's core motive is to close that gap: to give an individual student or job seeker the same kind of structural, keyword, and quality analysis that a large company's hiring software would perform, completely free, running locally, with no account, no paid API key, and no barrier to entry. This is why the project deliberately avoids relying on any paid or externally-hosted AI service — a resource that helps students shouldn't depend on whether an API key still works, whether a free tier is available in a particular country, or whether a company keeps offering free credits. By building the intelligence directly out of transparent, well-understood, open-source techniques — TF-IDF, sentence embeddings, and rule-based checks — the tool remains permanently usable, inspectable, and improvable by anyone who wants to build on it.

There is also a learning-focused motive behind how the project was built. It intentionally combines multiple layers of Natural Language Processing sophistication side-by-side — from simple keyword matching, to statistical TF-IDF scoring, to modern deep-learning-based semantic embeddings — specifically so that anyone studying the code can see, in one place, the evolution of how computers have learned to understand written language, and can directly compare how each technique performs on the exact same real-world problem.

9. A Beginner's Roadmap: How to Read This Project

If you are new to programming or data science and want to actually understand this project rather than just run it, here is a suggested order to study it in, and what to focus on at each step.

Step 1 — Start with `data/skills_list.py`

This is the simplest file in the whole project: just one variable holding a list of strings. Confirm you understand what a Python list is and how a value can be imported into another file before moving on.

Step 2 — Read `utils/extract_text.py` next

This file is short, has one function, and introduces the idea of using an external library (`pdfplumber`) to do something you couldn't easily do with plain Python alone. Notice the `'if page_text:'` safety check — try to imagine what would happen without it.

Step 3 — Move to the simpler functions in `utils/analyzer.py`

Read `clean_text()`, `extract_skills()`, `check_action_verbs()`, and `check_contact_info()` first. These are all short, use basic Python (loops, if-statements, list comprehensions), and don't require any machine learning background to follow.

Step 4 — Study `get_match_score()` and `get_semantic_score()`

These are the most conceptually advanced functions in the project. Don't worry about understanding the exact mathematics of TF-IDF or transformer embeddings on a first read — focus instead on the pattern: both functions take two pieces of text, turn them into numbers, and measure how close those numbers are. That core idea (comparing meaning by comparing numbers) is the foundation of almost all modern NLP.

Step 5 — Read `analyze_resume()` to see how everything connects

By this point you'll recognize every function being called inside it. Focus on how the results are combined into the three-pillar score (content, format, keyword) — this is the project's central design idea.

Step 6 — Finally, read `app.py`

Now that you understand what `analyze_resume()` produces, `app.py` will make much more sense — it's simply taking that dictionary of results and displaying each piece with the appropriate Streamlit component.

Step 7 — Run it, break it, and fix it

The single best way to truly learn this project is to intentionally change small things — alter a scoring weight, add a new skill to the list, change a chart's color — and observe what happens. Making small, deliberate changes and observing the results is how real programming fluency is built.

Closing note: Every file in this project was built to be readable on its own terms. There is no part of this codebase that requires advanced mathematics or years of experience to at least partially understand — and that accessibility was itself an intentional design goal, in keeping with the project's overall motive of making professional-grade resume analysis available to everyone, including beginners studying the very code that powers it.